



Implementation Core Spec — Classes, Database, Login, Wiring



Signatures, members, and step logic only. No full method bodies — you type those. Every item here is a fact you need to build the app, not advice.

Contents

1. Project file layout
2. Database schema (tables, columns, keys)
3. Model classes
4. DbContext
5. Identity: what exists, what you configure
6. Program.cs wiring (exact order)
7. TaskController: every action, signature + step logic
8. Views: what each file contains
9. Client-side validation wiring
10. Migration commands and order
11. Config files
12. Class relationship map
13. Build sequence (dependency-safe order)
14. Verification queries and tests

1. Project file layout

```
TaskManager/  
├─ Program.cs           # composition root: DI, middleware, routes  
├─ appsettings.json     # ConnectionStrings, Logging  
├─ appsettings.Development.json  
├─ TaskManager.csproj   # package references  
├─ Models/  
│   ├─ TaskItem.cs      # entity + data annotations  
│   ├─ ApplicationUser.cs # extends IdentityUser  
│   └─ ViewModels/  
│       ├─ TaskCreateViewModel.cs  
│       └─ TaskEditViewModel.cs
```

```

├── TaskListViewModel.cs
├── Data/
│   ├── ApplicationDbContext.cs           # DbSet<TaskItem>, OnModelCreating
│   └── SeedData.cs (optional)
├── Controllers/
│   ├── TaskController.cs                 # [Authorize], 9 actions
│   └── HomeController.cs
├── Views/
│   ├── Task/{Index,Create,Edit,Delete,Details}.cshtml
│   └── Shared/{_Layout,_ValidationScriptsPartial,_Alerts}.cshtml
├── Areas/Identity/Pages/Account/         # scaffolded Login, Register, Logout, Manage
├── Migrations/                           # generated, committed to git
├── wwwroot/
│   ├── js/task-validation.js
│   └── css/site.css

```

NuGet packages required

Package	Purpose
Microsoft.EntityFrameworkCore.Sqlite	SQLite provider (or .SqlServer)
Microsoft.EntityFrameworkCore.Tools	dotnet ef design-time
Microsoft.AspNetCore.Identity.EntityFrameworkCore	Identity stores
Microsoft.AspNetCore.Identity.UI	scaffolded Razor login/register pages
Microsoft.EntityFrameworkCore.Design	migrations

2. Database schema

2.1 Tables created by Identity (do not hand-edit)

Table	Purpose	Key columns you actually use
AspNetUsers	user accounts	Id (PK, TEXT/GUID string), UserName, NormalizedUserName, Email, NormalizedEmail, EmailConfirmed, PasswordHash, SecurityStamp, ConcurrencyStamp, LockoutEnd, AccessFailedCount, • DisplayName (yours)
AspNetRoles	roles	Id, Name, NormalizedName
AspNetUserRoles	user ↔ role join	UserId, RoleId
AspNetUserClaims	per-user claims	UserId, ClaimType, ClaimValue
AspNetUserLogins	external logins	LoginProvider, ProviderKey, UserId
AspNetUserTokens	tokens	UserId, LoginProvider, Name, Value
AspNetRoleClaims	role claims	RoleId, ClaimType
__EFMigrationsHistory	applied migrations	MigrationId, ProductVersion

2.2 Your table

Tasks

Column	SQL type (SQLite)	Constraints	Notes
<code>Id</code>	INTEGER	PK, AUTOINCREMENT	identity/sequence
<code>Title</code>	TEXT	NOT NULL, max 100	<code>[Required]</code> <code>[StringLength(100)]</code>
<code>Description</code>	TEXT	NULL, max 500	optional
<code>DueDate</code>	TEXT (ISO-8601)	NOT NULL	<code>DateTime</code> in C#
<code>IsCompleted</code>	INTEGER (0/1)	NOT NULL, default 0	<code>bool</code> in C#
<code>CreatedAt</code>	TEXT	NOT NULL, default now	set server-side
<code>CompletedAt</code>	TEXT	NULL	set when toggled complete
<code>Priority</code>	INTEGER	NOT NULL, default 1	enum: 0 Low, 1 Medium, 2 High
<code>UserId</code>	TEXT	NOT NULL, FK → <code>AspNetUsers.Id</code> , ON DELETE CASCADE	the ownership column

Indexes

Index	Columns	Why
<code>IX_Tasks_UserId</code>	<code>UserId</code>	every query filters on it
<code>IX_Tasks_UserId_IsCompleted</code>	<code>UserId</code> , <code>IsCompleted</code>	dashboard filter tabs
<code>IX_Tasks_UserId_DueDate</code>	<code>UserId</code> , <code>DueDate</code>	sorting/overdue queries

Relationship: `AspNetUsers` 1 —→ many `Tasks`. Principal = user, dependent = task. Delete behaviour = Cascade.

2.3 Reference DDL (what the migration will produce)

```
CREATE TABLE "Tasks" (
  "Id"          INTEGER NOT NULL CONSTRAINT "PK_Tasks" PRIMARY KEY AUTOINCREMENT,
  "Title"       TEXT     NOT NULL,
  "Description" TEXT     NULL,
  "DueDate"     TEXT     NOT NULL,
  "IsCompleted" INTEGER NOT NULL DEFAULT 0,
  "CreatedAt"   TEXT     NOT NULL,
  "CompletedAt" TEXT     NULL,
  "Priority"     INTEGER NOT NULL DEFAULT 1,
  "UserId"      TEXT     NOT NULL,
  CONSTRAINT "FK_Tasks_AspNetUsers_UserId"
    FOREIGN KEY ("UserId") REFERENCES "AspNetUsers" ("Id") ON DELETE CASCADE
);
CREATE INDEX "IX_Tasks_UserId" ON "Tasks" ("UserId");
```

3. Model classes

3.1 `Models/TaskItem.cs`

Member	Type	Attributes	Rule
<code>Id</code>	<code>int</code>	—	PK by convention
<code>Title</code>	<code>string</code>	<code>[Required(ErrorMessage=...)]</code> , <code>[StringLength(100, MinimumLength=3)]</code> , <code>[Display(Name="Task title")]</code>	trimmed before save
<code>Description</code>	<code>string?</code>	<code>[StringLength(500)]</code>	nullable
<code>DueDate</code>	<code>DateTime</code>	<code>[Required]</code> , <code>[DataType(DataType.Date)]</code> , <code>[Display(Name="Due date")]</code>	date only, no time
<code>IsCompleted</code>	<code>bool</code>	<code>[Display(Name="Completed")]</code>	default false
<code>CreatedAt</code>	<code>DateTime</code>	<code>[ScaffoldColumn(false)]</code>	set in Create action
<code>CompletedAt</code>	<code>DateTime?</code>	<code>[ScaffoldColumn(false)]</code>	null unless completed
<code>Priority</code>	<code>TaskPriority</code> (enum)	<code>[Required]</code>	enum stored as int
<code>UserId</code>	<code>string</code>	<code>[ValidateNever]</code> or excluded from binding	never bound from form
<code>User</code>	<code>ApplicationUser?</code>	<code>[ValidateNever]</code>	navigation property
<code>IsOverdue</code>	<code>bool</code> (computed)	<code>[NotMapped]</code>	<code>!IsCompleted &&</code> <code>DueDate.Date <</code> <code>DateTime.Today</code>

```
public enum TaskPriority { Low = 0, Medium = 1, High = 2 }
```

3.2 Models/ApplicationUser.cs

```
class ApplicationUser : IdentityUser
{
    string? DisplayName [StringLength(50)]
    DateTime CreatedAt
    ICollection<TaskItem> Tasks // inverse navigation
}
```

Inherited from `IdentityUser` (do not redeclare): `Id`, `UserName`, `Email`, `PasswordHash`, `PhoneNumber`, `EmailConfirmed`, `LockoutEnabled`, `SecurityStamp`, `TwoFactorEnabled`.

3.3 ViewModels (why: never bind the entity directly)

Class	Fields	Used by
<code>TaskCreateViewModel</code>	<code>Title</code> , <code>Description</code> , <code>DueDate</code> , <code>Priority</code>	Create GET/POST
<code>TaskEditViewModel</code>	<code>Id</code> , <code>Title</code> , <code>Description</code> , <code>DueDate</code> , <code>Priority</code> , <code>IsCompleted</code>	Edit GET/POST
<code>TaskListViewModel</code>	<code>IList<TaskItem> Tasks</code> , <code>string? Filter</code> , <code>string? SortBy</code> , <code>int PendingCount</code> , <code>int CompletedCount</code> , <code>int OverdueCount</code>	Index

Rule: ViewModels contain no `UserId`. Mapping entity ↔ ViewModel happens in the controller.

4. Data/ApplicationDbContext.cs

```
class ApplicationDbContext : IdentityDbContext<ApplicationUser>
{
    ctor(DbContextOptions<ApplicationDbContext> options) : base(options)
}
```

```

DbSet<TaskItem> Tasks { get; set; }

override void OnModelCreating(ModelBuilder builder)
    base.OnModelCreating(builder)           // REQUIRED – configures Identity tables
    builder.Entity<TaskItem>(e => {
        e.ToTable("Tasks")
        e.HasKey(t => t.Id)
        e.Property(t => t.Title).IsRequired().HasMaxLength(100)
        e.Property(t => t.Description).HasMaxLength(500)
        e.Property(t => t.IsCompleted).HasDefaultValue(false)
        e.Property(t => t.Priority).HasConversion<int>()
        e.HasOne(t => t.User)
            .WithMany(u => u.Tasks)
            .HasForeignKey(t => t.UserId)
            .OnDelete(DeleteBehavior.Cascade)
        e.HasIndex(t => t.UserId)
        e.HasIndex(t => new { t.UserId, t.IsCompleted })
    })

```

Failure modes

Mistake	Symptom
Forgot <code>base.OnModelCreating</code>	Identity tables missing / migration errors
Inherited <code>IdentityDbContext</code> without <code><ApplicationUser></code>	<code>DisplayName</code> never reaches DB
Changed model, no new migration	<code>SqliteException: no such column</code>

5. Identity: what exists vs what you configure

5.1 Services you inject

Service	Key members you call
<code>userManager<ApplicationUser></code>	<code>GetUserId(User)</code> , <code>GetUserAsync(User)</code> , <code>FindByIdAsync</code> , <code>CreateAsync(user, password)</code> , <code>AddClaimAsync</code>
<code>SignInManager<ApplicationUser></code>	<code>PasswordSignInAsync(email, pwd, isPersistent, lockoutOnFailure)</code> , <code>SignOutAsync()</code> , <code>IsSignedIn(User)</code>
<code>ClaimsPrincipal User</code> (on controller)	<code>User.Identity.IsAuthenticated</code> , <code>User.Identity.Name</code> , <code>User.FindFirstValue(ClaimTypes.NameIdentifier)</code>

Two ways to get the current user id — both valid:

```

_userManager.GetUserId(User)
User.FindFirstValue(ClaimTypes.NameIdentifier)

```

5.2 Login flow — mechanical steps

1. Anonymous request hits `[Authorize]` endpoint.

2. Authorization middleware issues 302 → `/Identity/Account/Login?ReturnUrl=/Task`.
3. Login page POSTs to its own handler → `SignInManager.PasswordSignInAsync(...)`.
4. Identity hashes the submitted password (PBKDF2) and compares to `PasswordHash`.
5. On success: builds `ClaimsPrincipal` with `NameIdentifier = user.Id`, serializes it, encrypts with Data Protection keys, writes cookie `.AspNetCore.Identity.Application`.
6. Redirect to `ReturnUrl`.
7. On each later request, cookie middleware decrypts the cookie and populates `HttpContext.User`.
8. Logout: `SignOutAsync()` deletes the cookie.

5.3 Identity options to set in `Program.cs`

Option	Typical value
<code>Password.RequiredLength</code>	6
<code>Password.RequireNonAlphanumeric</code>	false (dev convenience)
<code>Password.RequireUppercase</code> / <code>RequireDigit</code>	true
<code>SignIn.RequireConfirmedAccount</code>	false (no email sender configured)
<code>Lockout.MaxFailedAccessAttempts</code>	5
<code>Lockout.DefaultLockoutTimeSpan</code>	<code>TimeSpan.FromMinutes(5)</code>
<code>User.RequireUniqueEmail</code>	true

Cookie options: `ExpireTimeSpan = 30 min`, `SlidingExpiration = true`, `LoginPath = "/Identity/Account/Login"`, `AccessDeniedPath = "/Identity/Account/AccessDenied"`.

6. `Program.cs` wiring — exact order

```
// ----- SERVICES -----
builder.Configuration.GetConnectionString("DefaultConnection")
builder.Services.AddDbContext<ApplicationDbContext>(o => o.UseSqlite(conn))
builder.Services.AddDatabaseDeveloperPageExceptionFilter()
builder.Services
    .AddDefaultIdentity<ApplicationUser>(options => { /* section 5.3 */ })
    .AddEntityFrameworkStores<ApplicationDbContext>()
builder.Services.ConfigureApplicationCookie(o => { /* paths, expiry */ })
builder.Services.AddControllersWithViews()
builder.Services.AddRazorPages()           // REQUIRED for Identity UI

var app = builder.Build()

// ----- MIDDLEWARE (order is load-bearing) -----
1. if (env.IsDevelopment()) app.UseMigrationsEndPoint() else app.UseExceptionHandler("/Home/Error") + app.UseHsts()
2. app.UseHttpsRedirection()
3. app.UseStaticFiles()
4. app.UseRouting()
5. app.UseAuthentication()           // must precede UseAuthorization
6. app.UseAuthorization()
```

```

7. app.MapControllerRoute("default", "{controller=Home}/{action=Index}/{id?}")
8. app.MapRazorPages()           // Identity endpoints
9. app.Run()

```

Order failures

Wrong order	Result
<code>UseAuthorization</code> before <code>UseAuthentication</code>	always redirected to Login even when logged in
<code>UseRouting</code> after auth calls	endpoint metadata unavailable, <code>[Authorize]</code> ignored
Missing <code>AddRazorPages</code> / <code>MapRazorPages</code>	404 on <code>/Identity/Account/Login</code>
Missing <code>UseStaticFiles</code>	no CSS/JS → validation silently dead

7. Controllers/TaskController.cs

Class-level

```

[Authorize]
public class TaskController : Controller
{
    private readonly ApplicationDbContext _context
    private readonly UserManager<ApplicationUser> _userManager
    ctor(ApplicationDbContext, UserManager<ApplicationUser>) // DI
    private string CurrentUserId => _userManager.GetUserId(User)
    private Task<TaskItem?> FindOwnedTaskAsync(int id) // single ownership gate
}

```

`FindOwnedTaskAsync(int id)` step logic:

1. Query `_context.Tasks` where `Id == id && UserId == CurrentUserId`.
2. `FirstOrDefaultAsync()`.
3. Return result (null means either missing or not yours — caller returns `NotFound()`).

Every action

#	Signature	Verb / route	Step logic
1	<code>Task<IActionResult> Index(string? filter, string? sortBy)</code>	GET <code>/Task</code>	build <code>IQueryable</code> filtered by <code>UserId</code> → apply filter (<code>pending</code> / <code>completed</code> / <code>overdue</code>) → apply sort → <code>ToListAsync</code> → compute counts → return <code>View(TaskListViewModel)</code>
2	<code>Task<IActionResult> Details(int id)</code>	GET <code>/Task/Details/{id}</code>	<code>FindOwnedTaskAsync</code> → null ⇒ <code>NotFound()</code> → <code>View(task)</code>
3	<code>IActionResult Create()</code>	GET <code>/Task/Create</code>	return <code>View(new TaskCreateViewModel { DueDate = DateTime.Today })</code>
4	<code>Task<IActionResult> Create(TaskCreateViewModel vm)</code>	POST <code>[ValidateAntiForgeryToken]</code>	<code>ModelState.IsValid</code> ⇒ <code>View(vm)</code> → map to new <code>TaskItem</code> → set <code>UserId = CurrentUserId</code> , <code>CreatedAt =.UtcNow</code> , <code>IsCompleted = false</code> →

#	Signature	Verb / route	Step logic
			<code>Add</code> → <code>SaveChangesAsync</code> → <code>TempData["Success"]</code> → <code>RedirectToAction(nameof(Index))</code>
5	<code>Task<IActionResult> Edit(int id)</code>	GET <code>/Task/Edit/{id}</code>	<code>FindOwnedTaskAsync</code> → null ⇒ <code>NotFound()</code> → map to <code>TaskEditViewModel</code> → <code>View(vm)</code>
6	<code>Task<IActionResult> Edit(int id, TaskEditViewModel vm)</code>	POST <code>[ValidateAntiForgeryToken]</code>	<code>id != vm.Id</code> ⇒ <code>BadRequest()</code> → <code>ModelState.IsValid</code> ⇒ <code>View(vm)</code> → <code>FindOwnedTaskAsync</code> → null ⇒ <code>NotFound()</code> → copy allowed fields only → <code>SaveChangesAsync</code> → <code>TempData</code> → redirect Index
7	<code>Task<IActionResult> Delete(int id)</code>	GET <code>/Task/Delete/{id}</code>	<code>FindOwnedTaskAsync</code> → null ⇒ <code>NotFound()</code> → <code>View(task)</code> (confirmation)
8	<code>Task<IActionResult> DeleteConfirmed(int id)</code>	POST <code>/Task/Delete/{id}</code> <code>[ActionName("Delete")]</code> <code>[ValidateAntiForgeryToken]</code>	<code>FindOwnedTaskAsync</code> → null ⇒ <code>NotFound()</code> → <code>Remove</code> → <code>SaveChangesAsync</code> → <code>TempData</code> → redirect Index
9	<code>Task<IActionResult> ToggleComplete(int id)</code>	POST <code>/Task/ToggleComplete/{id}</code> <code>[ValidateAntiForgeryToken]</code>	<code>FindOwnedTaskAsync</code> → null ⇒ <code>NotFound()</code> → flip <code>IsCompleted</code> → set/clear <code>CompletedAt</code> → <code>SaveChangesAsync</code> → <code>TempData</code> → redirect Index

Non-negotiables

- Every by-Id action goes through `FindOwnedTaskAsync`. No exceptions.
- `UserId` is assigned only from `CurrentUserId`, never from `vm`.
- Every POST has `[ValidateAntiForgeryToken]`.
- Every mutation ends in `RedirectToAction`, never `View()` (Post-Redirect-Get).
- Reads use `AsNoTracking()`; writes do not.

Query snippets you will need (write them yourself, these are the shapes)

```

pending    : .Where(t => !t.IsCompleted)
completed  : .Where(t => t.IsCompleted)
overdue    : .Where(t => !t.IsCompleted && t.DueDate < DateTime.Today)
sort       : .OrderBy(t => t.IsCompleted).ThenBy(t => t.DueDate)
counts     : .CountAsync(t => !t.IsCompleted)

```

8. Views — required contents

File	Model directive	Must contain
<code>Views/Task/Index.cshtml</code>	<code>@model TaskListViewModel</code>	filter links, count badges, table rows, per-row Edit/Delete links + ToggleComplete form (POST + antiforgery),

File	Model directive	Must contain
		strikethrough for completed, badge for overdue, empty state
Views/Task/Create.cshtml	@model TaskCreateViewModel	<form asp-action="Create" method="post">, asp-for inputs, asp-validation-for spans, asp-validation-summary, @section Scripts { <partial name="_ValidationScriptsPartial" /> }
Views/Task/Edit.cshtml	@model TaskEditViewModel	same as Create + <input type="hidden" asp-for="Id" />
Views/Task/Delete.cshtml	@model TaskItem	read-only details + POST form with antiforgery
Views/Task/Details.cshtml	@model TaskItem	read-only fields
Views/Shared/_Alerts.cshtml	—	render TempData["Success"] / TempData["Error"] as dismissible Bootstrap alerts
Views/Shared/_Layout.cshtml	—	navbar with _LoginPartial, <partial name="_Alerts" />, jQuery + Bootstrap bundles, @await RenderSectionAsync("Scripts", required: false)

Tag helpers that matter: `asp-for`, `asp-action`, `asp-controller`, `asp-route-id`, `asp-validation-for`, `asp-validation-summary`, `asp-antiforgery`.

9. Client-side validation wiring

Chain: data annotation on model → `asp-for` tag helper emits `data-val-*` attributes → `jquery.validate.unobtrusive` reads them → blocks submit and writes message into `asp-validation-for` span.

Required scripts, in this order: `jquery.min.js` → `jquery.validate.min.js` → `jquery.validate.unobtrusive.min.js` → your `task-validation.js`.

Generated attribute example: `[Required][StringLength(100, MinimumLength=3)]` on `Title` → `data-val="true"`
`data-val-required="..." data-val-length data-val-length-max="100" data-val-length-min="3"`.

`wwwroot/js/task-validation.js` — things to implement

Behaviour	Hook
Disable Submit until Title non-empty	<code>\$('#Title').on('input', ...) toggling disabled</code>
Warn on past due date	<code>\$('#DueDate').on('change', ...) compare to today</code>
Live character counter for Description	<code>input</code> event + <code>maxlength</code>
Confirm before delete	<code>submit</code> handler with <code>confirm()</code>
Custom rule	<code>\$.validator.addMethod('notPastDate', fn, msg)</code> • <code>\$.validator.unobtrusive.adapters.addBool(...)</code>

Server-side is independent: `ModelState.IsValid` re-checks every annotation. Client validation off ⇒ server still rejects.

10. Migrations — commands and order

```
dotnet tool install --global dotnet-ef
```

```
# 1. Identity + TaskItem in one initial migration
```

```

dotnet ef migrations add InitialCreate
dotnet ef database update

# 2. after adding DisplayName to ApplicationUser
dotnet ef migrations add AddDisplayNameToUser
dotnet ef database update

# 3. after adding Priority / CreatedAt / CompletedAt
dotnet ef migrations add AddTaskMetadata
dotnet ef database update

# 4. after adding composite indexes
dotnet ef migrations add AddTaskIndexes
dotnet ef database update

# inspection / repair
dotnet ef migrations list
dotnet ef migrations script -o schema.sql
dotnet ef migrations remove # only if NOT yet applied
dotnet ef database update PreviousMigrationName # roll back
dotnet ef database drop -f # dev only

```

Rule: one migration per schema change, named after the change, committed to git. Never edit an applied migration — add a new one.

11. Config files

appsettings.json

```

{
  "ConnectionStrings": {
    "DefaultConnection": "Data Source=taskmanager.db"
  },
  "Logging": { "LogLevel": { "Default": "Information" } },
  "AllowedHosts": "*"
}

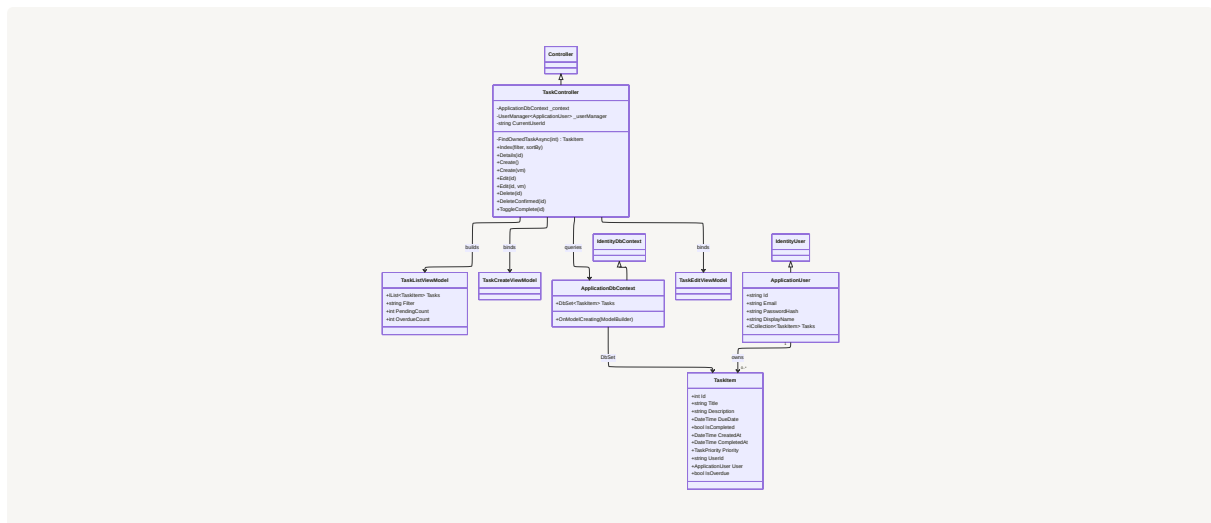
```

SQL Server alternative: "Server=

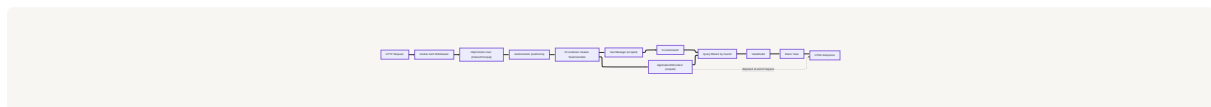
{localdb}\\mssqllocaldb;Database=TaskManager;Trusted_Connection=True;MultipleActiveResultSets=true" + UseSqlServer .

.gitignore must include: bin/ , obj/ , *.db , *.db-shm , *.db-wal , appsettings.Production.json .

12. Class relationship map



Runtime object graph per request



Lifetimes: `DbContext` = Scoped (one per request). `UserManager` / `SignInManager` = Scoped. `Controller` = Transient per request. Never store a `DbContext` in a static field.

13. Build sequence (dependency-safe)

Step	Build this	Cannot start before	Verify by
1	Project scaffold + packages	—	<code>dotnet run</code> shows template
2	<code>ApplicationUser</code>	1	compiles
3	<code>ApplicationDbContext</code> inherits <code>IdentityDbContext<ApplicationUser></code>	2	compiles
4	<code>Program.cs</code> DI + middleware	3	<code>/Identity/Account/Register</code> loads
5	<code>TaskItem</code> • enum	2	compiles
6	<code>DbSet<TaskItem></code> • <code>OnModelCreating</code>	3, 5	—
7	<code>InitialCreate</code> migration + update	6	<code>Tasks</code> • <code>AspNetUsers</code> exist in DB
8	ViewModels	5	compiles
9	<code>TaskController</code> skeleton + <code>FindOwnedTaskAsync</code>	7, 8	<code>/Task</code> returns 302 to Login
10	Index action + view	9	empty dashboard renders when logged in
11	Create GET/POST + view	10	task appears in DB and list
12	Edit + Delete + Details	11	full CRUD works
13	ToggleComplete	11	flag flips, <code>CompletedAt</code> set
14	Ownership checks audited on all by-Id actions	12, 13	cross-user URL test blocked

Step	Build this	Cannot start before	Verify by
15	Annotations + <code>task-validation.js</code>	11	invalid submit blocked client-side
16	TempData + <code>_Alerts</code> partial	11	alert after each action
17	Filters, sorting, counts, badges	10	filter links change results
18	Publish / Docker	all	published DLL serves app

14. Verification

14.1 SQL checks (SQLite browser or CLI)

```

.tables -- expect Tasks + 7 AspNet* + _
_EFMigrationsHistory
PRAGMA table_info('Tasks'); -- confirm columns/types
PRAGMA foreign_key_list('Tasks'); -- confirm FK to AspNetUsers
SELECT MigrationId FROM __EFMigrationsHistory; -- confirm all migrations applied
SELECT Id, Email FROM AspNetUsers; -- note two test user Ids
SELECT UserId, COUNT(*) FROM Tasks GROUP BY UserId; -- ownership distribution
SELECT * FROM Tasks WHERE UserId IS NULL; -- MUST return zero rows

```

14.2 Security test matrix

Test	Action	Pass condition
Anonymous access	GET <code>/Task</code> logged out	302 to Login
IDOR — read	User B GET <code>/Task/Details/{A's id}</code>	404/403
IDOR — edit GET	User B GET <code>/Task/Edit/{A's id}</code>	404/403
IDOR — edit POST	User B POSTs edit for A's id	404/403, DB unchanged
IDOR — delete POST	User B POSTs delete for A's id	404/403, row still present
IDOR — toggle POST	User B POSTs ToggleComplete for A's id	404/403
Mass assignment	Add <code>UserId=<A's id></code> field to B's Create form	task saved with B's id, not A's
CSRF	POST without antiforgery token	400
Server validation	Disable JS, submit empty Title	redispays form with error
PRG	Refresh after create	no duplicate row
Cascade	Delete a user	that user's tasks removed

14.3 Definition of done for the implementation core

- ☐ `Tasks.UserId` is `NOT NULL` with FK and index.
- ☐ All 9 controller actions exist with correct verbs and antiforgery tokens.
- ☐ All by-Id actions route through one ownership helper.
- ☐ `UserId` never appears in any ViewModel or form field.

- ☐ Middleware order matches Section 6 exactly.
- ☐ Every mutation redirects; none returns a View on success.
- ☐ Every migration in `Migrations/` is committed and applied.
- ☐ All 11 rows of the security matrix pass.



Single most important invariant: **every read and every write is scoped by `UserId` taken from the auth cookie, never from user input.**